

In One Sentence

“This system checks if a person is known by comparing their face with stored employee faces.”

Imagine a **smart security system at a company entrance**.

PART 1 - Registering an Employee (Teaching the System a Face)

Step 1: Employee uploads their picture

An employee opens the app (our React frontend) and uploads their photo.

Think of this like:

“Saving our face in the system so it knows us later”

Step 2: Photo is stored safely

The photo is sent and saved in **cloud storage (S3 bucket)**.

Like putting the picture in a **digital folder in the cloud**

Step 3: System analyses the face

A backend function (Lambda) automatically wakes up and sends the image to **Amazon Rekognition**.

Rekognition:

Looks at the face and creates a **unique facial fingerprint** (not the actual image, but data about the face)

Step 4: Save identity details

The system stores:

- Name (e.g., Larry Page)
- Face ID (from Rekognition)

inside a database (**DynamoDB**)

Like:

“Linking the face to a real person’s name”

End result of registration:

The system now *knows this person*

PART 2 - Authenticating a Person (Checking Who We Are)

Now someone (employee or visitor) comes in.

Step 1: Person uploads a new photo

They take/upload a picture using the app.

Like:

“Hey system, do you know me?”

Step 2: Request goes through API Gateway

The request goes through a gateway (like a receptionist) to the backend.

API Gateway =

“Front desk that routes requests to the right place”

Step 3: Backend checks the face

Another Lambda function takes the image and sends it to **Rekognition**

Rekognition now:

- Compares this new face
- Against all stored employee faces

Step 4: Matching happens

Rekognition gives a similarity score:

- High match → Same person
- Low match → Unknown person

Step 5: Get person details

If a match is found:

- System looks up the name in **DynamoDB**

Step 6: Send result back

The result goes back to the frontend:

-  “Welcome” (match found)
-  “Access denied” (no match)

Behind the Scenes (Invisible but Important)

- **IAM Roles** → control who can access what (security guard rules)
- **CloudWatch** → records logs (like CCTV for debugging)

Simple Real-Life Analogy

Think of it like:

<i>Real Life</i>	<i>Our System</i>
Security guard	API Gateway
Brain of the system	Lambda
Face recognition machine	Rekognition
Filing cabinet	DynamoDB
Photo storage	S3
User interface	React app

Final Explanation

“First, we register employees by uploading their photos. The system analyses their faces and stores their identity. Later, when someone uploads a new photo, the system compares it with stored faces and tells us if the person is recognised or not.”